# EXQUITECH — Agentic AI Internship

*Script Reference Guide*

## 1. Prompting Techniques

### 1.1 Zero-Shot Prompting

| | |
|---|---|
| **Script** | `01-prompting-techniques/zero_shot.py` |
| **Prompt** | `01-prompting-techniques/prompts/zero_shot.json` |

### 1.2 Few-Shot / In-Context Learning

| | |
|---|---|
| **Script** | `01-prompting-techniques/few_shot.py` |
| **Prompt** | `01-prompting-techniques/prompts/few_shot.json` |

### 1.3 Role Prompting

| | |
|---|---|
| **Script** | `01-prompting-techniques/role_prompting.py` |
| **Prompt** | `01-prompting-techniques/prompts/role_prompting.json` |

### 1.4 Chain Prompting

| | |
|---|---|
| **Script** | `01-prompting-techniques/chain_prompting.py` |
| **Prompt** | `01-prompting-techniques/prompts/chain_prompting.json` |

### 1.5 Chain-of-Thought (CoT)

| | |
|---|---|
| **Script** | `01-prompting-techniques/chain_of_thought.py` |
| **Prompt** | `01-prompting-techniques/prompts/chain_of_thought.json` |

### 1.6 Self-Consistency

| | |
|---|---|
| **Script** | `01-prompting-techniques/self_consistency.py` |
| **Prompt** | `01-prompting-techniques/prompts/self_consistency.json` |

### 1.7 Tree-of-Thought

| | |
|---|---|
| **Script** | `01-prompting-techniques/tree_of_thought.py` |
| **Prompt** | `01-prompting-techniques/prompts/tree_of_thought.json` |

### 1.8 ReAct Prompting

| | |
|---|---|
| **Script** | `01-prompting-techniques/react_prompting.py` |
| **Prompt** | `01-prompting-techniques/prompts/react_prompting.json` |

### 1.9 Structured Output

| | |
|---|---|
| **Script** | `01-prompting-techniques/structured_output.py` |
| **Prompt** | `01-prompting-techniques/prompts/structured_output.json` |

### 1.10 Prompt Templates (LangChain)

| | |
|---|---|
| **Script** | `01-prompting-techniques/prompt_templates.py` |
| **Prompt** | `01-prompting-techniques/prompts/prompt_templates.json` |

## 2. Single Agent Patterns

### 2.1 Basic ReAct Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/basic_react_agent.py` |
| **MAF** | `single-agents/maf/basic_react_agent.py` |

### 2.2 Web Search Agent (Tavily)

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/react_agent_websearch.py` |
| **MAF** | `single-agents/maf/react_agent_websearch.py` |

### 2.3 File Search / RAG Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/rag_react_agent.py` |
| **MAF** | `single-agents/maf/rag_react_agent.py` |
| **Vector DB** | `utility/faiss_index/` (built via `utility/build_faiss.py`) |

### 2.4 In-Memory RAG Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/in-memory-rag.py` |
| **MAF** | `single-agents/maf/in-memory-rag.py` |

### 2.5 Code Interpreter Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/code-interpreter.py` |
| **MAF** | `single-agents/maf/code-interpreter.py` |

### 2.6 Multi-Tool Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/multiple-tools-agent-langgraph.py` |
| **LangGraph+** | `single-agents/langgraph/multiple-tools-graph.py` |
| **MAF** | `single-agents/maf/multiple-tools-agent.py` |

### 2.7 Skills & MCP Demo

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/mcp_skills_langgraph.py` |
| **MAF** | `single-agents/maf/mcp_skills_maf.py` |
| **MCP Server** | `utility/mcp-server.py` |

### 2.8 CSV Analyzer Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/CSV-File-Analyzer.py` |
| **MAF** | `single-agents/maf/CSV-File-Analyzer.py` |

### 2.9 SQLite Query Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/sql_react_agent.py` |
| **MAF** | `single-agents/maf/sql_react_agent.py` |
| **Database** | `utility/chinook.db` |

## 3. Agent Design Patterns

### 3.1 Reflection Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/Reflection-Agent.py` |
| **MAF** | `single-agents/maf/Reflection-Agent.py` |

### 3.2 Reflexion Agent

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/Reflexion_Agent.py` |
| **MAF** | `single-agents/maf/Reflexion_Agent.py` |
| **Memory** | `single-agents/langgraph/reflexion_memory/  (FAISS index)` |

### 3.3 Agentic RAG

| | |
|---|---|
| **LangGraph** | `single-agents/langgraph/Agentic_Rag.py` |
| **MAF** | `single-agents/maf/Agentic_Rag.py` |

### 3.4 Supervisor Agent

| | |
|---|---|
| **LangGraph Router** | agent-patterns/Langgraph/Router.py |
| **LangGraph State** | agent-patterns/Langgraph/routerstate.py |
| **LG Chat Agent** | agent-patterns/Langgraph/chat_agent.py |
| **LG Code Agent** | agent-patterns/Langgraph/code_agent.py |
| **LG Knowledge** | agent-patterns/Langgraph/knowledge_agent.py |
| **LG Engineering** | agent-patterns/Langgraph/engineering_agent.py |
| **MAF Router** | agent-patterns/MAF/Router.py |
| **MAF Chat** | agent-patterns/MAF/chat_agent.py |
| **MAF Code** | agent-patterns/MAF/code_agent.py |
| **MAF Knowledge** | agent-patterns/MAF/knowledge_agent.py |
| **MAF Engineering** | agent-patterns/MAF/engineering_agent.py |

### 3.5 Handoffs & Router

| | |
|---|---|
| **LangGraph** | agent-patterns/Langgraph/Router.py (routing logic) |
| **MAF** | agent-patterns/MAF/Router.py (routing logic) |
| **State** | agent-patterns/Langgraph/routerstate.py |

### 3.6 Parallelization

| | |
|---|---|
| **LangGraph** | agent-patterns/Langgraph/Router.py (async parallel nodes) |
| **MAF** | agent-patterns/MAF/Router.py (async/await dispatch) |

### 3.7 Human-in-the-Loop (HITL)

| | |
|---|---|
| **LangGraph** | agent-patterns/Langgraph/code_agent.py (interrupt / resume) |
| **LangGraph** | agent-patterns/Langgraph/Router.py (SqliteSaver checkpoint) |
| **Checkpoints** | agent-patterns/Langgraph/code_checkpoints.db |
| **MAF** | agent-patterns/MAF/code_agent.py |

## 4. Memory & Context

### 4.1 Short-Term Memory

| | |
|---|---|
| **LangGraph** | MemorySaver() — used in all single-agents/langgraph/ scripts |
| **MAF** | In-session state via AzureOpenAIResponsesClient |

### 4.2.1 Long-Term Memory Vector-Based

| | |
|---|---|
| **FAISS Index** | utility/faiss_index/index.faiss + index.pkl |
| **Builder** | utility/build_faiss.py |
| **Inspector** | utility/see-vector-store.py |
| **Reflexion Memory** | single-agents/langgraph/reflexion_memory/ |

### 4.2.2 Long-Term Memory State-Based SQLite

| | |
|---|---|
| **SQLite(Router)** | agent-patterns/Langgraph/router_checkpoints.db |
| **SQLite(Code)** | agent-patterns/Langgraph/code_checkpoints.db |
| **SQLite(Engineering)** | agent-patterns/Langgraph/engineering_checkpoints.db |

### 4.2.3 Long-Term Memory JSON-Based

| | |
|---|---|
| **MAF Code Memory** | agent-patterns/MAF/memory/code_memory.json |
| **MAF Eng. Memory** | agent-patterns/MAF/memory/engineering_memory.json |
| **MAF Router Memory** | agent-patterns/MAF/memory/router_memory.json |

## 4.3 LangGraph Checkpointing

| | |
|---|---|
| **Router DB** | agent-patterns/Langgraph/router_checkpoints.db |
| **Code DB** | agent-patterns/Langgraph/code_checkpoints.db |
| **Chat DB** | agent-patterns/Langgraph/chat_checkpoints.db |
| **Router Log** | agent-patterns/Langgraph/router_log.txt |
| **Code Log** | agent-patterns/Langgraph/code_supervisor_log.txt |
| **Chat Log** | agent-patterns/Langgraph/chat_supervisor_log.txt |

# 01 Basic React Agent

This script creates a simple AI assistant that can answer weather questions and basic questions about AI, Python, or LangGraph. The user types a message like “What is AI?” or “What is the weather in SF?”, and the assistant decides which tool to use, runs it, and returns the result. It also uses MemorySaver to keep conversation memory during the session.

### LangGraph

- State defined with TypedDict; messages annotated with add_messages so nodes append instead of overwrite
- Tools created with @tool decorator and wrapped in a ToolNode
- Explicit graph with nodes and edges; LLM bound to tools decides when to call them

### MAF

- Plain Python functions used as tools — no plugin class or @tool decorator needed
- Tools passed directly to the agent: tools=[get_weather, ...]
- No explicit graph or shared state — the LLM handles flow internally

### Comparison

| | **LangGraph** | **MAF** |
|---|---|---|
| **Graph & State** | Explicit graph, TypedDict state, add_messages | No graph — LLM manages flow internally |
| **Tool Setup** | @tool decorator + ToolNode required | Plain Python functions, no decorator |
| **Boilerplate** | More — nodes, edges, state wiring | Significantly less |
| **Result Quality** | Same for simple tasks | Same for simple tasks |

**Verdict:** *Both produce identical output quality. The difference is orchestration and visibility, not results.*

# 02 Web Search Agent

**This script creates an AI assistant that can decide whether to answer directly or search the web first. If the question is about news, recent facts, or something uncertain, it uses the search tool and then returns the result. It also prints debugging information showing what decision was made and what the tool returned.**

### LangGraph

- Same graph structure as Basic React Agent — State, nodes, conditional edges
- Web search tool (e.g. Tavily) wrapped in ToolNode
- Results automatically appended to shared state via add_messages

### MAF

- Web search added as a plain Python function tool
- Passed directly to the agent — no graph wiring needed
- LLM handles when to call it internally

### Comparison

| | **LangGraph** | **MAF** |
|---|---|---|
| **Setup** | Graph overhead: State, nodes, edges | No graph — plain function tool |
| **Result passing** | Appended via add_messages automatically | LLM manages results internally |
| **Code complexity** | More boilerplate | Significantly less |
| **Result quality** | Same | Same |

**Verdict:** *Same result quality. LangGraph adds structure that does not improve output for this use case.*

# 03 ReAct Agent with File Search (RAG)

**This script creates an AI assistant that answers questions using information stored in a local knowledge base. When the user asks a question, the assistant first searches the stored documents for the most relevant information, then uses that context to generate an answer. If the answer is not found in the stored content, it replies with "I don't know."**

### LangGraph

- Pre-built FAISS vector store with Azure embeddings (text-embedding-3-large)
- Two explicit nodes: retrieve injects results as SystemMessage; agent node answers using context
- Graph flow: START → retrieve → agent → END — retrieval is guaranteed, hardwired as first node
- MemorySaver for conversation checkpointing across turns

### MAF

- Same FAISS setup and Azure embeddings
- Retrieval wrapped in a plain function rag_search(query) passed as a tool
- Agent instructed via prompt to ALWAYS call rag_search before answering
- No graph, no nodes — LLM decides based on instructions; no conversation memory

### Comparison

| | **LangGraph** | **MAF** |
|---|---|---|
| **Retrieval guarantee** | Guaranteed — hardwired first node | Not guaranteed — depends on LLM instructions |
| **Memory** | MemorySaver — built-in multi-turn memory | No conversation memory |
| **Retrieval quality** | Same (same FAISS index) | Same (same FAISS index) |
| **Boilerplate** | More — two explicit nodes | Less — single agent with one tool |

**Verdict:** *LangGraph edge here — retrieval is structurally guaranteed. MAF retrieval depends on the LLM following instructions.*

# 04 ReAct Agent with In-Memory RAG

This script creates an AI assistant that can answer questions only from uploaded files. The user can load PDF, TXT, or CSV documents, and the script reads them, splits them into smaller chunks, and stores them for searching. When the user asks a question, the assistant searches the uploaded documents for relevant information and answers only from that content. If the answer is not found in the documents, it clearly says so

**LangGraph**

- Ephemeral in-memory FAISS — no pre-built index, built at runtime from uploaded files
- load_document() handles PDF, TXT, CSV — chunks with RecursiveCharacterTextSplitter (500 / 50)
- Graph: START → agent → (conditional) → tools → agent → END
- RAG is NOT guaranteed — conditional edge means LLM decides whether to call the tool
- MemorySaver for multi-turn memory; CLI supports /load <file_path> at runtime

**MAF**

- Identical load_document() and search_documents() logic — same chunking, same FAISS store
- search_documents passed as a plain function — no @tool decorator
- Agent instructed via prompt to ALWAYS call search_documents first
- RAG is NOT guaranteed — LLM may skip if confident in its own knowledge
- No graph, no memory

**Comparison**

| | **LangGraph** | **MAF** |
|---|---|---|
| **RAG guaranteed?** | No — conditional edge, LLM decides | No — prompt instruction, LLM may skip |
| **Control mechanism** | tool_router function in code | System prompt rules |
| **Memory** | MemorySaver built-in | No memory |
| **RAG logic** | Identical underneath | Identical underneath |

**Verdict:** *Neither guarantees RAG — for different reasons. Both achieve the same result quality.*

# 05 ReAct Agent with Code Interpreter

This script creates an AI assistant that can run Python code to solve problems like math calculations, data analysis, transformations, and plotting. Instead of calculating manually, it writes code, executes it, and returns the result. It can also save charts and retry if the code fails

**LangGraph**

- execute_python: writes code to temp file, runs in sandboxed subprocess with 30s timeout
- Auto-injects matplotlib Agg backend preamble for plot saving without a display
- Retry loop: error appended to shared state, graph routes back to agent as a NEW LLM call
- Each retry is a separate graph event — can inspect or interrupt between retries
- MemorySaver for multi-turn memory

**MAF**

- Identical execute_python logic — same sandbox, same matplotlib preamble; timeout 15s
- No @tool decorator — plain function passed directly
- Retry loop: LLM reads tool output and retries WITHIN the same single run
- Retries are invisible — only the final output is visible
- No graph, no memory

**Comparison**

| | **LangGraph** | **MAF** |
|---|---|---|
| **Retry mechanism** | Multiple separate LLM calls cycling through graph | Single LLM run — retries handled internally |
| **State between retries** | Shared via add_messages | LLM holds context internally |
| **Visibility** | Full — each retry is a separate graph event | None — only final output visible |
| **Interruption?** | Yes — can inspect between retries | No — no access between retries |
| **Result quality** | Same | Same |

**Verdict:** *Same result quality. LangGraph gives visibility between retries. MAF is simpler but opaque.*

# 06 Multiple Tools Agent with Fallbacks

This script creates a multi-tool AI assistant that can search the web, answer questions from uploaded files, run Python code, and provide weather information. It automatically decides which tool to use based on the user's request and includes retry logic if a tool fails.

**LangGraph**

- 4 tools: web_search, search_documents, execute_python, get_weather — all @tool decorated
- Extended State with tool_retry_counts (per tool) and error_loop_count (global safety cap)
- Dedicated error_tracker node runs after every ToolNode — scans for failures, increments counters
- MAX_RETRIES = 3 per tool call, MAX_ERROR_LOOPS = 6 global cap
- Every new fallback path requires a new node + new edge + updated router logic

**MAF**

- Same 4 tools + simple_search as extra fallback — all plain Python functions
- @with_retries decorator handles low-level retries at the function level — clean and reusable
- Entire fallback chain defined in plain English instructions
- LLM reads [TOOL ERROR] prefix and follows the fallback chain naturally
- Adding a new fallback = one instruction line

**Comparison**

| | **LangGraph** | **MAF** |
|---|---|---|
| **A fails → try B** | New node + new edge | One instruction line |
| **A fails → try B → try C** | 2 nodes + 3 edges | Two instruction lines |
| **Adding a new fallback** | Restructure the graph | Add one instruction line |
| **Fallback guaranteed?** | Yes — wired into graph | No — LLM may skip under heavy context |
| **Retry logic** | error_tracker node — significant boilerplate | @with_retries decorator — clean and reusable |

**Verdict:** *MAF wins significantly on fallbacks — simpler, more flexible, easier to extend. LangGraph wins only on guarantee of execution.*

# 07 Reflexion Agent

This script creates a Reflexion-style AI assistant that answers a question, critiques its own answer, suggests web searches, and then revises the answer multiple times. It also stores past mistakes in a vector database so future answers can improve by avoiding similar issues.

**LangGraph**

- 3 explicit nodes: draft, tools, revise
- Fixed flow: START → draft → tools → revise → (loop) → tools → revise → END
- Draft node: LLM writes first answer, critiques itself, suggests search queries
- ResponderWithRetries retries up to 3x inside the node if JSON format is wrong
- Every step guaranteed — hardwired edges, LLM cannot skip draft, search, or revise
- Loop controlled by MAX_ITERATIONS = 5; FAISS vector memory stores past reflections

**MAF**

- Single agent with one tool: run_queries
- In one LLM run: draft → critique → search queries → call run_queries → revise → return final JSON
- Outer Python retry loop checks reflection after each run — rebuilds prompt if needed
- Search is NOT guaranteed — LLM decides whether to call run_queries
- Same FAISS vector memory for past reflections

**Comparison**

| | **LangGraph** | **MAF** |
|---|---|---|
| **LLM calls per turn** | 3 separate calls — one per node | 1 call — does everything internally |
| **Step guarantees** | Guaranteed via hardwired edges | Not guaranteed — LLM follows instructions |
| **Visibility** | Full — each step is a graph event | None — only final JSON visible |
| **Retry loop** | Structural — built into graph | Manual Python loop — you build it yourself |
| **Memory** | Same FAISS implementation | Same FAISS implementation |

**Verdict:** *Both are equal for a 3-step Reflexion flow. LangGraph's real advantage appears at scale — many agents, parallel execution, checkpointing.*

# 08 MCP Agent (Model Context Protocol)

This script creates an AI assistant that can connect to external MCP tools through a server. The assistant decides when to use those tools, runs them, and returns the results to the user. It also handles tool errors and retries if needed.

**LangGraph**

- Connects to MCP server via SSE (sse_client at http://127.0.0.1:8787/sse)
- load_mcp_tools(session) — automatically loads all tools as LangChain-compatible objects in one line
- Custom tool_node loops through tool_calls, invokes each, appends ToolMessage results to state
- Error handling inside tool_node — catches exceptions and returns as ToolMessage

**MAF**

- Same SSE connection to MCP server
- MCP tools do not come as plain Python functions — MAF needs real function signatures to build LLM tool schemas
- make_mcp_tool_func() dynamically wraps each MCP tool: builds inspect.Signature from JSON schema
- Significant wrapper boilerplate just to make MCP tools compatible with MAF

**Comparison**

| | **LangGraph** | **MAF** |
|---|---|---|
| **MCP tool loading** | load_mcp_tools() — one line, tools ready | make_mcp_tool_func() — full custom wrapper |
| **Native adapter** | Yes — LangChain MCP adapter | No — manual signature injection required |
| **Boilerplate** | Minimal | Significant |
| **Result** | Same | Same |

**Verdict:** *LangGraph wins — native MCP adapter makes integration trivial. MAF requires substantial manual wrapping to achieve the same.*

# 09 CSV File Analyzer

---

**LangGraph**

- 4 tools with @tool: load_csv, get_csv_info, query_csv, visualize_csv
- Global _dataframes dict stores loaded DataFrame — shared across tool calls
- query_csv uses eval() for pandas expressions; visualize_csv uses exec() for matplotlib
- State manually maintained across CLI loop and passed to graph.invoke() each turn

**MAF**

- Identical 4 tools as plain Python functions — same eval() and exec() logic
- No @tool decorator needed
- Instruction forces get_csv_info() at the start of every message to check if CSV is loaded
- No graph, no state management, no node wiring

**Comparison**

| | **LangGraph** | **MAF** |
|---|---|---|
| **Tool setup** | @tool decorator + ToolNode | Plain functions — no decorator |
| **State management** | Explicit per turn via graph.invoke() | Implicit within agent.run() |
| **Forced tool call** | No — LLM decides | Yes — instruction forces get_csv_info() every turn |
| **Tool logic** | Identical underneath | Identical underneath |
| **Result** | Same analysis and charts | Same analysis and charts |

**Verdict:** *Identical results. MAF is significantly less code for the same functionality.*

# 10 Reflection Agent (Essay Writer)

---

This script creates an essay-writing assistant that generates an essay, critiques it like a teacher, and then rewrites it several times to improve quality before returning the final version.

**LangGraph**

- Two separate LLM chains: generate (essay writer) and reflect (critic), each with their own ChatPromptTemplate
- Key trick: reflection node flips message roles — AI messages become HumanMessages and vice versa
- Graph: START → generate → (conditional) → reflect → generate → ...
- should_continue router counts messages — after 6 messages (3 full loops) it routes to END
- MemorySaver for conversation memory

**MAF**

- Two separate agents: generator_agent and reflector_agent — each with their own instructions
- No role-flipping needed — each agent has a clear role and Python loop passes content explicitly
- Outer Python loop manually orchestrates: generate → reflect → revise × 3
- Each round passes previous essay + critique + original request as a combined prompt

**Key Differences**

LangGraph's role-flipping trick is clever but adds mental overhead — MAF avoids it entirely by using two separate agents with clear roles.

LangGraph loop is controlled structurally via message count in the router. MAF loop is a simple Python for loop. Both do exactly 3 rounds of generate → reflect → revise.

**Verdict:** *MAF is significantly simpler and easier to reason about.*

# 11 SQL Agent

### LangGraph

- Two tools: get_schema and query_sqlite — both @tool decorated, wrapped in ToolNode
- Basic SQL protection: blocks DROP, DELETE, UPDATE inside the tool
- Graph: START → agent → (conditional) → tools → agent → END
- MemorySaver for multi-turn memory — agent remembers schema from previous turns

### MAF

- Identical get_schema and query_sqlite logic — same SQLite connection, same protection
- No @tool decorator — plain functions passed directly
- No graph, no memory

### Comparison

| | **LangGraph** | **MAF** |
|---|---|---|
| **Tool logic** | Completely identical to MAF | Completely identical to LangGraph |
| **Multi-turn memory** | MemorySaver — remembers schema across questions | No memory |
| **Pattern** | get schema → write SQL → execute → return result | get schema → write SQL → execute → return result |
| **Code amount** | More — graph + memory boilerplate | Less — same result per turn |

**Verdict:** *LangGraph has a meaningful edge with multi-turn memory. MAF is less code for the same per-turn result.*

# 12 Agentic RAG

The system checks whether to search the knowledge base, retrieves related documents, reviews their relevance, and if they are not good enough, rewrites the query and tries again. Once sufficient relevant information is found, it generates a final answer.

**LangGraph**

- Multiple explicit nodes: retrieve → grade docs → rewrite question → answer → optionally retry
- Conditional edges: relevant docs → answer node; irrelevant docs → rewrite node; failed answer → retry retrieve
- Graph state stores: messages, retrieved docs, rewritten question, grading results, final answer
- MemorySaver/checkpointer can preserve conversation and retrieval state across turns
- Easy to inspect which node failed and why
- Easy to extend with human approval, retries, parallel retrieval, web fallback, or multiple retrievers

**MAF**

- One main agent with tools: vector search, web search, grading instructions
- The LLM internally decides: should I retrieve? rewrite? search again? answer now?
- No explicit graph or node separation — easier to build, most logic stays in the prompt
- Can still do retrieval, rewriting, grading, and retries — but all inside one agent loop
- Harder to debug and harder to add complex branching or human approval flows

**Comparison**

| | **LangGraph** | **MAF** |
|---|---|---|
| **Step visibility** | Full — each step is a separate node and state field | Hidden — the whole flow is inside one agent call |
| **Debugging** | Easy — see what docs were retrieved, why rejected, when query was rewritten | Hard — only final output is visible |
| **Complex branching** | Excellent — retries, fallback, memory, parallel retrieval, human-in-the-loop | Limited — harder to add complex branching |
| **Code volume** | More | Less |
| **Result quality** | Same for straightforward flows | Same for straightforward flows |

**Verdict:** *LangGraph is much better when the RAG pipeline becomes complex. MAF is better when you want a simpler implementation with less code.*

# 13 Multi-Agent Supervisor

Both systems route a user message to the right agent, pause for human approval, and either run code or return an answer. The difference is in how they achieve it.

There is a router that will

## LangGraph

The flow is built as a graph with nodes and edges.

When the graph needs human input it calls interrupt(), which saves the entire state to SQLite at that exact moment and suspends. When you respond, the graph resumes from exactly where it stopped. If the process crashes while waiting, nothing is lost — SQLite has everything.

Parallelism is a structural decision: returning a list from the routing function triggers LangGraph to fan out and run all destinations simultaneously. No async knowledge needed.

Langgraph also gives us automatic checkpoiting example:

```
sqlite_conn = sqlite3.connect("router_checkpoints.db",
check_same_thread=False)
checkpointer = SqliteSaver(sqlite_conn)
graph = workflow.compile(checkpointer=checkpointer)
```

This stores graph state automatically:

- messages
- current node
- pending interrupts
- approvals
- final answer
- active supervisor
- anything inside the state object

This is useful for:

- pause/resume
- conversation memory
- graph recovery
- debugging
- keeping agent state across sessions

## MAF

The flow is built manually with normal Python functions, loops, and if/else statements instead of a graph with nodes and edges.

When the workflow needs user approval or human input, we need to save the current state ourselves in SQLite, Redis, JSON, or memory. Then when the user responds, we reload that state and continue from the correct function manually. If the process crashes while waiting, recovery only works if we already built that logic ourselves.

Parallel execution is also manual. Usually we use `asyncio.gather()` to run multiple agents or tools together, then manually combine their outputs back into the state.

MAF does not automatically checkpoint anything. We decide what to save, for example:

- messages
- current supervisor
- pending approval

- generated code
- tool outputs
- final answer
- current step

This gives more flexibility and full control, but it also means more logic, more state management, more debugging, and more work to support pause/resume and crash recovery.

| Topic | LangGraph | MAF / Notes |
|---|---|---|
| State Management | Strong advantage — state is built into the framework. Each node automatically receives and updates the same shared state object. | Must manually design and manage state (tables, rows, columns, serialization, loading). |
| State Persistence | Can persist state directly through a checkpointer (e.g. SQLite) — no manual serialization or context restoration needed. | Requires manually designing persistence logic, restoring execution context, and loading previous messages. |
| Debugging | Easier — explicit graph structure shows exactly which node ran, what state entered/exited, which edge was taken, and where failures occurred. | Less visibility into execution flow; harder to trace failures. |
| Shared State Between Nodes | All nodes share the same state. One node updates fields (e.g. research_results, generated_code, final_answer) and the next node automatically sees them. | Shared state must be explicitly passed or managed manually. |
| Routing / Control Flow | Cleaner — define graph edges and conditional transitions declaratively. | Requires building large if/else chains manually. |
| Parallel Execution (Sync Nodes) | Easier — can fan out into multiple branches automatically through graph edges. | Must orchestrate parallelism manually. |
| Parallel Execution (Async Tools) | No automatic parallelism — asyncio.gather() or manual async orchestration still required for tools like Tavily, FAISS, web APIs, async DB calls. | Same requirement — asyncio.gather() or manual async orchestration. |
| Parallel Advantage Summary | Strongest for regular synchronous node execution, not when nodes contain async tool calls. | Comparable when async tooling is involved. |

### The Core Trade-off

| | LangGraph | MAF |
|---|---|---|
| **Crash recovery** | SQLite saves state at every node — survives crashes | No persistence — state is lost if process dies |
| **Web / API ready** | Yes — interrupt() designed for HTTP request input | No — input() only works in terminals |
| **Parallelism** | Automatic — return a list from router | Manual — asyncio.gather, easy to get wrong silently |
| **Memory** | Managed by graph state and SQLite automatically | Manual — JSON files, easy to forget saving |
| **Simplicity** | More boilerplate, a framework to learn | Just Python — easy to debug and change |

**Verdict:** *The core logic is identical. The difference is where complexity lives — in graph wiring (LangGraph) or in your own code (MAF).*

# Final Assessment

---

| Criterion | LangGraph | MAF |
|---|---|---|
| **Ease of Setup** | Moderate — graph, state, nodes, edges required | Easy — plain functions, minimal config |
| **Learning Curve** | Steep — explicit graph concepts | Gentle — close to standard Python |
| **Tool Integration** | @tool decorator + ToolNode | Direct plain functions, no decorator |
| **Multi-Agent Support** | Built-in — native supervisors, routers | Manual — requires custom orchestration |
| **Memory Management** | Built-in — add_messages, MemorySaver | Manual — no built-in memory |
| **Fallback Handling** | New node + edge per fallback | Plain English instructions — trivial to extend |
| **Step Guarantees** | Strong — hardwired edges enforce flow | Weak — depends on LLM following instructions |
| **Visibility / Debug** | Full — every step a separate graph event | Limited — only final output visible |
| **MCP Integration** | Native adapter — one line | Manual wrapper — significant boilerplate |
| **Result Quality** | Same for simple tasks | Same for simple tasks |
| **Production Readiness** | High — checkpointing, resume, parallel exec | Medium — simpler workflows |
| **Best For** | Complex multi-agent, stateful workflows | Simple agents, fast prototyping |

---

**When to choose LangGraph**

- You need pause and resume that survives a crash — graph saves state at every node to SQLite
- You are building for an API or web app — interrupt() is designed for HTTP request input, not terminal input()
- Your workflow is complex with many branching paths, loops, and conditional edges
- You want parallelism for free — returning a list from a routing function is all it takes
- You want state managed for you — every node reads from and writes to a shared state dict

**When to choose MAF**

- You want simplicity — it is just Python, a loop, and async functions, nothing to learn
- You are building a terminal tool and iterating fast — easy to change, easy to debug
- Your workflow is straightforward — a single execution path with no complex branching
- You are comfortable building memory yourself — you own the JSON files, the session logic, the summarization

---

**The honest bottom line:** *MAF can do everything LangGraph does if you are willing to build it yourself. LangGraph gives you the hard parts — persistence, pause and resume, parallelism, state management — for free, but at the cost of more boilerplate and a framework to learn. The question is always whether the complexity of your problem justifies the complexity of the framework.*